

Exemplo de derivação sintática: for em Java

Linguagem escolhida

Java.

Fonte da gramática consultada

Java Language Specification (JLS), Java SE 26 Edition, seção 14.14 – “The for Statement” (capítulo 14: Blocks, Statements, and Patterns). A gramática abaixo foi simplificada para fins didáticos, preservando a estrutura sintática essencial do comando for.

Código que será derivado

```
public class ExemploFor {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println(i);  
        }  
    }  
}
```

Apesar de o programa completo ser apresentado como exemplo válido e executável, a derivação será concentrada no seguinte trecho:

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

Regras de produção selecionadas

Para concentrar a derivação no comando de repetição, foi usada uma gramática reduzida. Os símbolos entre < > são não terminais; os demais representam terminais da linguagem.

<for_statement> ::= for (<for_init> ; <condition> ; <for_update>) <statement>

<for_init> ::= <local_variable_declaration>

<local_variable_declaration> ::= <type> <identifier> = <integer_literal>

<type> ::= int

<condition> ::= <identifier> <= <integer_literal>

<for_update> ::= <identifier> ++

<statement> ::= <block>

<block> ::= { <statement_list> }

<statement_list> ::= <expression_statement>

<expression_statement> ::= <method_invocation> ;

<method_invocation> ::= System.out.println (<expression>)

<expression> ::= <identifier> | <integer_literal>

<identifier> ::= i

<integer_literal> ::= 1 | 5

Derivação passo a passo

A derivação abaixo começa no símbolo não terminal <for_statement> e substitui, passo a passo, cada não terminal até obter a sequência de terminais correspondente ao trecho desejado.

```
<for_statement>
⇒ for ( <for_init> ; <condition> ; <for_update> ) <statement>
⇒ for ( <local_variable_declaration> ; <condition> ; <for_update> ) <statement>
⇒ for ( <type> <identifier> = <integer_literal> ; <condition> ; <for_update> ) <statement>
⇒ for ( int <identifier> = <integer_literal> ; <condition> ; <for_update> ) <statement>
⇒ for ( int i = <integer_literal> ; <condition> ; <for_update> ) <statement>
⇒ for ( int i = 1 ; <condition> ; <for_update> ) <statement>
⇒ for ( int i = 1 ; <identifier> <= <integer_literal> ; <for_update> ) <statement>
⇒ for ( int i = 1 ; i <= <integer_literal> ; <for_update> ) <statement>
⇒ for ( int i = 1 ; i <= 5 ; <for_update> ) <statement>
⇒ for ( int i = 1 ; i <= 5 ; <identifier> ++ ) <statement>
⇒ for ( int i = 1 ; i <= 5 ; i ++ ) <statement>
⇒ for ( int i = 1 ; i <= 5 ; i ++ ) <block>
⇒ for ( int i = 1 ; i <= 5 ; i ++ ) { <statement_list> }
⇒ for ( int i = 1 ; i <= 5 ; i ++ ) { <expression_statement> }
⇒ for ( int i = 1 ; i <= 5 ; i ++ ) { <method_invocation> ; }
⇒ for ( int i = 1 ; i <= 5 ; i ++ ) { System.out.println ( <expression> ) ; }
⇒ for ( int i = 1 ; i <= 5 ; i ++ ) { System.out.println ( <identifier> ) ; }
⇒ for ( int i = 1 ; i <= 5 ; i ++ ) { System.out.println ( i ) ; }
```

Forma concreta em Java, com pontuação:

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

Identificação dos terminais e não terminais

Terminais

Símbolos que aparecem efetivamente no código após o término da derivação:

```
for
(
)
int
i
=
1
;
<=
5
```

```
++  
{  
}  
System.out.println  
(  
)  
;  
;
```

Não terminais

Símbolos que representam estruturas sintáticas substituídas ao longo da derivação:

```
<for_statement>  
<for_init>  
<local_variable_declaration>  
<type>  
<identifier>  
<integer_literal>  
<condition>  
<for_update>  
<statement>  
<block>  
<statement_list>  
<expression_statement>  
<method_invocation>  
<expression>
```

Ao final da derivação, nenhum não terminal permaneceu: restou apenas uma sequência de símbolos terminais correspondente ao trecho de código Java escolhido.